

Data Analysis Code- HW 2

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from scipy.stats import chi2_contingency

#downloading files
data1_OG = pd.read_csv('Responses 1 - Fardina.csv')
data2_OG = pd.read_csv('Responses 2 - Max.csv')
data3 = pd.read_csv('Responses 2 - Fardina.csv')
data4 = pd.read_csv('Responses 1 - Max.csv')

#Renaming some columns for standardization
data1 = data1_OG.rename(columns = {
    'How would you rate your religiousness / spirituality?':
    'How would you rate your religiousness?'})
data2 = data2_OG.rename(columns = {
    'How would you rate your religiousness / spirituality?':
    'How would you rate your religiousness?'})

#Half are labelled OG and other half have swapped genders
#for easier interpretability
OG_data = pd.concat([data1, data2], ignore_index = True)
Swapped_data = pd.concat([data3, data4], ignore_index = True)

#combine all data
all_data = pd.concat([OG_data, Swapped_data], ignore_index=True)

#Clean up response columns to be numerical
def clean_response(response):
```

```

if response == "Strongly a jerk":
    return 2
elif response == "Mildly a jerk":
    return 1
elif response == "Not a jerk":
    return 0
else:
    return np.nan

#Mapping religion / spirituality to numbers for consistency regardless of
#whether the columns say spirituality or religion
def religious_mapping(value):
    if isinstance(value, str):
        if "not" in value.lower():
            return -2
        elif "somewhat" in value.lower():
            return 0
        elif "strongly" in value.lower():
            return 2
        else:
            return np.nan
    else:
        return np.nan

all_data['How would you rate your religiousness?'] = (
    all_data['How would you rate your religiousness?'].map(religious_mapping))

response_columns = [col for col in all_data.columns if "jerk" in col]
for col in response_columns:
    all_data[col] = all_data[col].apply(clean_response)

# Create a gender column to indicate if the data had swapped genders or not.
all_data['gender_swap'] = [0] * len(OG_data) + [1] * len(Swapped_data)

```

```

#Clean up gender column
all_data['What bests represents your gender?'] = (
    all_data['What bests represents your gender?'].replace('Famale', 'Female'))

#Clean up age column
all_data['How old are you?'] = (
    pd.to_numeric(all_data['How old are you?'], errors='coerce'))

#Clean up Year column
all_data['What year are you?'] = (
    all_data['What year are you?'].replace('Sophmore', 'Sophomore'))

#Plotting based on gender
original_columns = OG_data.columns.tolist()
swapped_columns = Swapped_data.columns.tolist()

# Ensure the columns are matched correctly and plot
min_length = min(len(original_columns), len(swapped_columns))

for i in range(7, min_length):
    original_col = original_columns[i]
    swapped_col = swapped_columns[i]

    plt.figure(figsize=(15, 6))

    # Original Question Box Plot
    plt.subplot(1, 2, 1)
    sns.boxplot(x='What bests represents your gender?',
        y=original_col, data=all_data)
    plt.title(f"Original: {original_col}")

    # Swapped Question Box Plot
    plt.subplot(1, 2, 2)
    sns.boxplot(x='What bests represents your gender?',
        y=swapped_col, data=all_data)

```

```

plt.title(f"Swapped: {swapped_col}")

plt.tight_layout()
plt.show()

#Graphing question 7 specifically
original_q7_column = None
swapped_q7_column = None

for col in all_data.columns:
    if "My parents want us to come out for their anniversary" in col:
        if "sister" in col:
            original_q7_column = col
        elif "brother" in col:
            swapped_q7_column = col

if original_q7_column and swapped_q7_column:
    original_q7_data = all_data[[original_q7_column,
        'What bests represents your gender?']]
    swapped_q7_data = all_data[[swapped_q7_column,
        'What bests represents your gender?']]

    original_q7_data = original_q7_data.dropna()
    swapped_q7_data = swapped_q7_data.dropna()

if original_q7_column and swapped_q7_column:
    plt.figure(figsize=(15, 6))

    # Visualizing the original question 7
    plt.subplot(1, 2, 1)
    sns.boxplot(x='What bests represents your gender?',
        y=original_q7_data[original_q7_column], data=original_q7_data)
    plt.title(f"Original Genders")
    plt.xlabel("What is your gender")

```

```

plt.ylabel("Jerk Ranking")

# Visualizing the swapped question 7
plt.subplot(1, 2, 2)
sns.boxplot(x='What bests represents your gender?',
            y=swapped_q7_data[swapped_q7_column], data=swapped_q7_data)
plt.title(f"Swapped Genders")
plt.xlabel("What is your gender")
plt.ylabel("Jerk Ranking")

plt.tight_layout()
plt.show()

#Combined Analysis for gender_swap analysis
original_q7_data['gender_swap'] = 0
swapped_q7_data['gender_swap'] = 1

combined_q7_data = pd.concat(
    [original_q7_data.rename(columns={original_q7_column: 'response'}),
     swapped_q7_data.rename(columns={swapped_q7_column: 'response'})],
    ignore_index = True)

plt.figure(figsize=(10, 6))
sns.boxplot(x='gender_swap', y='response', data=combined_q7_data)
plt.title(f"Responses by Gender Swap: Question 7")
plt.xticks([0, 1], ['Consistent Genders', 'Swapped Genders'])
plt.show()

#Hypothesis test
group1 = combined_q7_data[combined_q7_data['gender_swap'] == 0]['response']
group2 = combined_q7_data[combined_q7_data['gender_swap'] == 1]['response']

stat, p1 = stats.ttest_ind(group1, group2)
test_type = "T-test"

stat, p2 = stats.ttest_ind(group2, group1, alternative='less')
test_type = "One-tailed T-test (less)"

```

```
#Did both one-tailed and two-tailed t-test
print(f"Question 7 Gender Swap Comparison: {test_type}, p-value = {p1}")
print(f"Question 7 Gender Swap Comparison: {test_type}, p-value = {p2}")
```

```
#Box and Whisker plots for Religiousness
i=1
for col in response_columns:
    plt.figure(figsize=(10, 6))
    sns.boxplot(x='How would you rate your religiousness?',
                y=col, data=all_data, palette='viridis')
    plt.title(f'Question {i} vs. Religiousness')
    plt.xlabel('Religiousness Level')
    plt.ylabel('Jerk Rating')
    plt.xticks(rotation=45, ha='right')
    plt.show()
    i+=1
```

```
#Chi^2 test for religion
i = 1
adjusted_alpha = 0.05 / len(response_columns)
religiousness_col = 'How would you rate your religiousness?'
for col in response_columns:
    contingency_table = pd.crosstab(all_data[religiousness_col], all_data[col])
    chi2, p, _, _ = chi2_contingency(contingency_table)
    if p < adjusted_alpha:
        print(f"Chi-square for Question {i}: Chi2 = {chi2:.3f}, P = {p:.3f},"
              "Significant")
    else:
        print(f"Chi-square for Question {i}: Chi2 = {chi2:.3f}, P = {p:.3f},"
              "Insignificant")

    i+=1
```

```
#Box and whisker plot for political standing
```

```

i = 1
for col in response_columns:
    plt.figure(figsize=(10, 6))
    sns.boxplot(x='You could describe yourself as...',
                y=col, data=all_data, palette='viridis')
    plt.title(f'Question {i} vs. Politics')
    plt.xlabel('Political standing')
    plt.ylabel('Jerk Rating')
    plt.xticks(rotation=45, ha='right')
    plt.show()
    i+=1

#Box and whisker plot for political standing
i = 1
adjusted_alpha = 0.05 / len(response_columns)
political_col = 'You could describe yourself as...'
for col in response_columns:
    contingency_table = pd.crosstab(all_data[political_col], all_data[col])
    chi2, p, _, _ = chi2_contingency(contingency_table)
    if p < adjusted_alpha:
        print(f"Chi-square for Question {i}: Chi2 = {chi2:.3f}, P = {p:.3f},"
              "Significant")
    else:
        print(f"Chi-square for Question {i}: Chi2 = {chi2:.3f}, P = {p:.3f},"
              "Insignificant")

    i+=1

```